Stack (LIFO) และการประยุกต์ใช้งาน

Data Structures & Object-Oriented Programming



Table of Contents

- Stack คืออะไร
- Operations ของ Stack และความซับซ้อน
- Bracket Matching
- หลักการ Undo/Redo
- โค้ดของ Node กับ Stack
- Application of stack

ทำไมต้องมี Stack?

พิจารณาสถานการณ์ 4 แบบต่อไปนี้ เป็นสิ่งที่พบได้ในโปรแกรมและระบบที่ใช้งานกันอยู่ทั่วไป

- **สถานการณ์ที่ 1 — การกด Undo ในโปรแกรมแก้ไขโค้ด**
  - เขียนโค้ดไปหลายบรรทัดแล้วพบว่าเขียนผิดพลาด การกด Ctrl+Z ครั้งแรกจะย้อนกลับไปยังผลลัพธ์ก่อนหน้าล่าสุดเสมอ และหากกด Ctrl+Z ต่อไปอีกครั้ง โปรแกรมจะย้อนกลับไปยังผลลัพธ์ก่อนหน้านั้นอีกทีหนึ่ง
  - การแก้ไขครั้งล่าสุดจะถูกยกเลิกก่อนเสมอ ไม่ใช่การแก้ไขครั้งแรกสุดที่เคยทำไว้
- **สถานการณ์ที่ 2 — การกดปุ่ม Back ในเว็บเบราว์เซอร์**
  - สมมติว่าผู้ใช้เปิดหน้าเว็บตามลำดับ A แล้วไป B แล้วไป C แล้วไป D
  - เมื่อกดปุ่ม Back เบราว์เซอร์จะพาผู้ใช้กลับไปยังหน้า C ไม่ใช่หน้า A เพราะหน้าที่เข้าชมล่าสุดคือหน้าที่ควรถูกย้อนกลับไปก่อน

ทำไมต้องมี Stack?

พิจารณาสถานการณ์ 4 แบบต่อไปนี้ เป็นสิ่งที่พบได้ในโปรแกรมและระบบที่ใช้งานกันอยู่ทั่วไป

- **สถานการณ์ที่ 3 — การตรวจสอบวงเล็บในโค้ดโปรแกรม**
  - พิจารณาโค้ดที่มีข้อผิดพลาดต่อไปนี้
    - def foo((bar) {):
  - เครื่องมือแก้ไขโค้ดสามารถตรวจพบได้ว่าโค้ดนี้มีข้อผิดพลาดเรื่องวงเล็บ
  - คำถามคือเครื่องมือนี้ทราบได้อย่างไร คำตอบคือมันต้องมีวิธีจดจำว่า “มีวงเล็บใดเปิดค้างไว้บ้าง” และ “วงเล็บที่เปิดล่าสุดจะต้องถูกปิดก่อนวงเล็บอื่นเสมอ”

ทำไมต้องมี Stack?

พิจารณาสถานการณ์ 4 แบบต่อไปนี้ เป็นสิ่งที่พบได้ในโปรแกรมและระบบที่ใช้งานกันอยู่ทั่วไป

- **สถานการณ์ที่ 4 — การทำงานของฟังก์ชันที่เรียกซ้อนกันใน Python**

```
def a():
    b()

def b():
    c()

def c():
    print("hello")

a()
```

- คำถาม คือ เมื่อฟังก์ชัน c() ทำงานเสร็จสิ้นแล้ว
- Python ทราบได้อย่างไรว่าต้องกลับไปทำงานต่อที่ฟังก์ชัน b()
  - ไม่ใช่กลับไปที่ฟังก์ชัน a() โดยตรง

รูปแบบร่วมของทั้งสี่สถานการณ์

- แม้ทั้งสี่สถานการณ์จะดูแตกต่างกันโดยสิ้นเชิงในแง่ของ domain ที่เกี่ยวข้อง แต่ทั้งหมดมีหลักการร่วมกันเพียงหนึ่งเดียวคือ สิ่งที่ถูกเพิ่มเข้ามาล่าสุด จะเป็นสิ่งที่ถูกนำออกไปก่อนเสมอ
- หากลองใช้ Python List ธรรมดาในการแก้ปัญหานี้ อาจเขียนโค้ดได้ในลักษณะดังนี้

```
history = []
history.append("state_1")
history.append("state_2")
history.append("state_3")

# ต้องการ state ล่าสุด
last = history[len(history) - 1]    # หรือ history[-1]
history.pop()
```

- **โค้ดชุดนี้ทำงานได้ถูกต้อง แต่มีปัญหาสำคัญประการหนึ่งซ่อนอยู่ นั่นคือ Python List เป็นโครงสร้างข้อมูลที่มีความสามารถมากเกินความจำเป็นสำหรับปัญหานี้**
- **ไม่มีสิ่งใดจำกัดไม่ให้โค้ดส่วนอื่นในโปรแกรมเขียน history[0] เพื่อดึงข้อมูลจากตำแหน่งต้นออกไปโดยตรง หรือแม้แต่ใช้ history.insert(...) แทรกข้อมูลลงกลาง list ได้อย่างอิสระ**
- **หากเกิดเหตุการณ์เช่นนี้ขึ้นโดยไม่ตั้งใจ พฤติกรรมที่ควรจะเป็นไปตามกฎ LIFO จะผิดเพี้ยนไปทันทีโดยไม่มีสัญญาณเตือนใดๆ**

Stack

- คือโครงสร้างข้อมูลที่ถูกออกแบบมาเพื่อแก้ปัญหานี้โดยเฉพาะ
- ด้วยการจำกัดความสามารถของตัวเองให้เหลือเพียง “เพิ่มและนำออกจากปลายด้านเดียวกันเท่านั้น”
  - ไม่มีช่องทางอื่นใดให้เข้าถึงข้อมูลได้เลย
- การจำกัดความสามารถเช่นนี้ อาจฟังดูเหมือนเป็นข้อเสีย แต่ในความเป็นจริงแล้วคือจุดแข็งที่สุดของ Stack
  - ทำให้พฤติกรรมของโครงสร้างข้อมูลถูกต้องตามที่ต้องการเสมอ
  - โดยอาศัยการออกแบบ (correct by design) ไม่ต้องพึ่งพาวินัยของผู้เขียนโค้ดในการใช้งานให้ถูกต้อง

คำศัพท์ที่ใช้เรียกตำแหน่งใน Stack

- ตำแหน่งบนสุดของ Stack ซึ่งเป็นตำแหน่งเดียวที่สามารถเพิ่มหรือนำข้อมูลออกได้ เรียกว่า **top**
- ส่วนตำแหน่งล่างสุด (ซึ่งเป็นข้อมูลที่ถูกเพิ่มเข้ามาเป็นตัวแรกสุด และจะเป็นตัวสุดท้ายที่ถูกนำออก) เรียกว่า **bottom**
- ทุก operation ของ Stack จะเกิดขึ้นที่ตำแหน่ง top เท่านั้นเสมอ

Stack คืออะไร

หลักการ LIFO

- Stack เป็นโครงสร้างข้อมูลที่ทำงานตามหลักการที่เรียกว่า LIFO (Last In, First Out)
  - แปลตรงตัวได้ว่า “สิ่งที่เข้ามาหลังสุด จะถูกนำออกก่อน”
- หลักการนี้คือคำตอบของทั้งสี่สถานการณ์ที่กล่าวไว้ในหัวข้อก่อนหน้า
  - การ undo, การกด back, การจับคู่วงเล็บ, หรือการเรียกฟังก์ชันซ้อนกัน

การเปรียบเทียบ: กองจานในร้านอาหาร

- เพื่อให้เห็นภาพของหลักการ LIFO ได้ชัดเจนยิ่งขึ้น ลองจินตนาการถึงกองจานที่วางซ้อนกันอยู่ในร้านอาหาร
  - จานที่ถูกวางซ้อนไว้ล่าสุด (ซึ่งอยู่บนสุดของกอง) จะเป็นจานที่ถูกหยิบไปใช้งานก่อนเสมอ
  - ไม่มีพนักงานคนใดพยายามดึงจานออกจากกลางกองหรือจากด้านล่างสุด เพราะการกระทำเช่นนั้นจะทำให้กองจานทั้งหมดล้มลง
  - ข้อจำกัดทางกายภาพนี้เองที่บังคับให้พฤติกรรมของกองจานเป็นไปตามหลัก LIFO โดยธรรมชาติ

การเปรียบเทียบ: กองจานในร้านอาหาร

- พิจารณาลำดับเหตุการณ์ต่อไปนี้เพื่อให้เห็นภาพชัดเจนยิ่งขึ้น

```
วางจาน A    →  [A]
วางจาน B    →  [A, B]        ← B อยู่บนสุด
วางจาน C    →  [A, B, C]     ← C อยู่บนสุด

หยิบจาน     →  ได้ C  (ออกก่อน เพราะเข้ามาหลังสุด)
หยิบจาน     →  ได้ B
หยิบจาน     →  ได้ A
```

Stack ในทางโปรแกรมมิ่งก็ทำงานด้วยหลักการเดียวกันนี้ทุกประการ

เพียงแต่แทนที่จะเป็นจาน สิ่งที่ถูกจัดเก็บคือข้อมูลใดๆ ก็ตามที่โปรแกรมต้องการ

Operations ของ Stack และความซับซ้อน

Operations ของ Stack และความซับซ้อน

- Stack ในฐานะโครงสร้างข้อมูลเชิงนามธรรม (Abstract Data Type หรือ ADT)
  - มี operation หลักเพียง 4 ตัวเท่านั้น
  - ทุกตัวมีความซับซ้อนแบบ O(1) ทั้งหมด

Operation	ความหมาย	Complexity
`push(item)`	เพิ่ม `item` ลงบนตำแหน่ง `top`	O(1)
`pop()`	นำ `item` ที่อยู่บนตำแหน่ง `top` ออก แล้วคืนค่ากลับมา	O(1)
`peek()`	ดูค่า `item` ที่อยู่บนตำแหน่ง `top` โดยไม่นำออกจาก Stack	O(1)
`is_empty()`	ตรวจสอบว่า Stack ว่างเปล่าอยู่หรือไม่	O(1)

Stack ไม่มี operation ใดที่เข้าถึงข้อมูลที่ตำแหน่งกลาง ไม่มีการค้นหา (search) ไม่มีการแทรกหรือลบข้อมูลที่ตำแหน่งใดๆ นอกจาก top และไม่มีการเข้าถึงข้อมูลผ่าน index เหมือนที่ Array หรือ Dynamic Array ทำได้

การเลือก Implementation, Function Call Stack, และหลักการ Bracket Matching

แนวทางในการ Implement Stack

- Stack เป็นเพียงโครงสร้างข้อมูลเชิงนามธรรม (ADT) ที่นิยาม operation และพฤติกรรมไว้เท่านั้น ส่วนการ implement จริงเบื้องหลังสามารถเลือกใช้โครงสร้างข้อมูลพื้นฐานได้มากกว่าหนึ่งแบบ
- ไม่ว่าจะเลือกแบบใด ผู้ใช้งานคลาส Stack จากภายนอกจะไม่รู้สึกถึงความแตกต่างใดๆ เพราะ interface (push, pop, peek, is_empty) ยังคงเหมือนเดิมทุกประการ
    - นี่คือหลักการของการเขียนโปรแกรมเชิงวัตถุที่เรียกว่า encapsulation
- แนวทางที่ 1 Array-based Stack
    - ใช้ Python List (หรือ DynamicArray) เป็นโครงสร้างเบื้องหลัง โดยกำหนดให้ตำแหน่ง top ของ Stack ตรงกับตำแหน่ง**ท้ายสุด**ของ List เสมอ
        - push จึงตรงกับ list.append() และ pop จึงตรงกับ list.pop()
- แนวทางที่ 2 Linked List-based Stack
    - การใช้ Linked List เป็นโครงสร้างเบื้องหลัง โดยกำหนดให้ตำแหน่ง top ของ Stack ตรงกับตำแหน่ง **head** ของ Linked List เสมอ

การเชื่อมโยงจาก SinglyLinkedList สู่ Stack

- มองว่า Stack คือ “การนำ SinglyLinkedList มาใช้งานโดยจำกัดสิทธิ์การเข้าถึงให้เหลือเฉพาะที่ head”
- ตารางแสดงการเทียบเคียง method ที่คุ้นเคยอยู่แล้วกับ operation ใหม่ของ Stack

Operation ของ Stack	เทียบเท่ากับ method ใดใน `SinglyLinkedList`	เหตุผล
`push(item)`	`prepend(item)`	เพิ่มข้อมูลใหม่ที่ `head` โดยไม่ต้อง traverse ตรงกับหลักการ push ที่ต้องเพิ่มที่ `top` ในเวลา O(1) พอดี
`pop()`	ค่าเทียบเท่ากับ `delete` ที่ตำแหน่ง head	การลบที่ `head` ของ Singly Linked List เป็น O(1) เพราะไม่ต้องหา `prev` เหมือนกรณีลบ Node อื่น จึงเหมาะสมพอดีกับความต้องการของ `pop`
`peek()`	การอ่านค่า `_head.data` โดยตรง โดยไม่เปลี่ยนแปลงค่า `_head`	เป็นเพียงการอ่านค่า ไม่มี method ใดใน `SinglyLinkedList` ที่ตรงกับสิ่งนี้โดยตรง เพราะเดิมทีเราไม่เคยจำกัดตัวเองให้อ่านได้แค่ที่ head เท่านั้น
`is_empty()`	เทียบเท่ากับการตรวจสอบว่า `_head is None`	ตรงกับเงื่อนไข edge case ที่เคยเรียนไว้ว่า list ว่างเปล่าเมื่อ `_head` เป็น `None`

เนื่องจาก Stack ไม่จำเป็นต้องเข้าถึง Node ที่ปลายอีกด้านหนึ่งเลย
การ implement Stack จาก Linked List จึงไม่จำเป็นต้องมี _tail pointer

Function Call Stack

- เมื่อฟังก์ชันที่เรียกซ้อนกันทำงานเสร็จสิ้น Python ทราบได้อย่างไรว่าต้องกลับไปทำงานต่อที่จุดใด
- คำตอบคือ Python มีกลไกที่เรียกว่า Call Stack ทำงานอยู่เบื้องหลังตลอดเวลาโดยที่ผู้เขียนโค้ดไม่ต้องจัดการเอง

```
def calculate_total(items):
    subtotal = sum_items(items)        # Python push frame ของ sum_items
    tax = calculate_tax(subtotal)      # Python push frame ของ calculate_tax
    return subtotal + tax

def sum_items(items):
    return sum(items)                  # เมื่อ return → Python pop frame ออก

def calculate_tax(amount):
    rate = get_tax_rate()              # Python push frame ของ get_tax_rate
    return amount * rate

def get_tax_rate():
    return 0.07                        # เมื่อ return → Python pop frame ออก
```

- ทุกครั้งที่มีการเรียกฟังก์ชันหนึ่งจากภายในอีกฟังก์ชันหนึ่ง Python จะสร้าง stack frame ขึ้นมาหนึ่งชุด ซึ่งเก็บข้อมูลของการเรียกฟังก์ชันนั้น (เช่น ตัวแปร local และตำแหน่งที่ต้องกลับไปทำงานต่อ) แล้ว push frame นั้นลงบน Call Stack
- เมื่อฟังก์ชันทำงานเสร็จสิ้นและมีการ return ค่า Python จะ pop frame นั้นออกจาก Call Stack แล้วกลับไปทำงานต่อที่ frame ซึ่งอยู่ถัดลงไป (ซึ่งก็คือฟังก์ชันที่เรียกมันมานั่นเอง)

Function Call Stack (2)

- หากลองพิจารณาสถานะของ Call Stack ในขณะที่ calculate_total([100, 200]) กำลังทำงานอยู่ และฟังก์ชัน get_tax_rate() กำลัง execute อยู่พอดี จะได้ภาพดังนี้

```
[ขณะที่ get_tax_rate() กำลังทำงาน]

TOP    →  get_tax_rate      (กำลัง execute)
          calculate_tax     (รอ get_tax_rate เสร็จ)
          calculate_total   (รอ calculate_tax เสร็จ)
BOTTOM →  <module>          (โปรแกรมหลัก)
```

- เมื่อ get_tax_rate() return ค่าออกมา frame ของมันจะถูก pop ออกจาก Call Stack ทันที ทำให้ calculate_tax() กลายเป็น frame ที่อยู่บนสุดและทำงานต่อไปได้
- เมื่อ calculate_tax() return ค่าออกมาอีกที frame ของมันก็จะถูก pop ออกไปเช่นกัน เหลือเพียง calculate_total() ทำงานต่อจนเสร็จสมบูรณ์

Function Call Stack (3)

- กลไกนี้เองที่อธิบายได้ว่าทำไมการเขียนฟังก์ชันแบบ recursive ที่ลึกเกินไปจึงทำให้เกิดข้อผิดพลาด
  - RecursionError: maximum recursion depth exceeded
- เพราะ Python จำกัดจำนวน frame สูงสุดที่ Call Stack สามารถเก็บไว้พร้อมกันได้ (โดยค่า default อยู่ที่ประมาณ 1,000 frames)
  - เพื่อป้องกันไม่ให้โปรแกรมใช้หน่วยความจำจนหมดจากการเรียกฟังก์ชันซ้อนกันแบบไม่มีที่สิ้นสุด
  - ปรากฏการณ์นี้เรียกว่า **stack overflow**

Bracket Matching

Bracket Matching

- หนึ่งใน application ที่แสดงให้เห็นถึงความเหมาะสมของ Stack ได้ชัดเจนที่สุด คือ การตรวจสอบว่าวงเล็บในโค้ดหรือนิพจน์ทางคณิตศาสตร์ถูกเปิดและปิดอย่างถูกต้องครบถ้วนหรือไม่ (เรียกว่าปัญหา bracket matching หรือ balanced parentheses)
- การพิจารณาว่าลำดับของวงเล็บใดๆ ถูกต้องหรือไม่ ต้องอาศัยกฎสามข้อต่อไปนี้ร่วมกัน
  - **กฎข้อที่ 1** วงเล็บเปิดทุกตัวต้องมีวงเล็บปิดชนิดเดียวกันมาจับคู่ด้วยเสมอ
  - **กฎข้อที่ 2** วงเล็บปิดที่พบ ต้องปิดวงเล็บเปิดตัวล่าสุดที่ยังค้างอยู่เท่านั้น ไม่สามารถข้ามไปปิดวงเล็บเปิดที่เก่ากว่าได้
  - **กฎข้อที่ 3** เมื่ออ่านข้อความจนจบแล้ว จะต้องไม่มีวงเล็บเปิดใดหลงเหลือค้างอยู่เลย
- ข้อสังเกต กฎข้อที่ 2 คือหลักการ LIFO นั่นเอง
  - วงเล็บเปิดที่ “เข้ามาล่าสุด” (ถูกเปิดล่าสุด) จะต้องถูก “นำออกก่อน” (ถูกปิดก่อน) ซึ่งตรงกับพฤติกรรมตามธรรมชาติของ Stack

แนวคิดของ Algorithm ตรวจสอบวงเล็บ

- เริ่มต้นด้วยการสร้าง stack ที่ว่างเปล่าขึ้นมาหนึ่งตัว
- จากนั้นอ่านข้อความทีละตัวอักษรตั้งแต่ต้นจนจบ
  - หากตัวอักษรที่อ่านพบเป็นวงเล็บเปิด ให้ push วงเล็บนั้นลงใน stack ทันที
  - แต่หากเป็นวงเล็บปิด ต้องตรวจสอบสองกรณี
    - กรณีแรกคือหาก stack ว่างเปล่าอยู่พอดี แสดงว่าไม่มีวงเล็บเปิดใดค้างอยู่ให้จับคู่ด้วยเลย จึงสรุปได้ทันทีว่าข้อความนี้ไม่ถูกต้อง
    - กรณีที่สองคือหาก stack ไม่ว่างเปล่า ให้ pop วงเล็บที่อยู่บนสุดออกมา แล้วตรวจสอบว่าวงเล็บที่ pop ออกมานั้นเป็นชนิดที่ตรงกับวงเล็บปิดที่กำลังพิจารณาอยู่หรือไม่
      - หากไม่ตรงกัน แสดงว่าข้อความนี้ไม่ถูกต้อง
- เมื่ออ่านข้อความจนครบทุกตัวอักษรแล้ว ขั้นตอนสุดท้ายคือการตรวจสอบว่า stack ว่างเปล่าหรือไม่
  - หาก stack ว่างเปล่า แสดงว่าวงเล็บเปิดทุกตัวได้ถูกจับคู่และปิดไปหมดแล้ว จึงสรุปว่าข้อความนี้ถูกต้อง
  - แต่หาก stack ยังมีวงเล็บหลงเหลืออยู่ แสดงว่ามีวงเล็บเปิดบางตัวที่ไม่เคยถูกปิดเลย จึงสรุปว่าข้อความนี้ไม่ถูกต้อง

ตัวอย่างการไล่ทำงานทีละขั้นตอน

ตัวอย่างที่ 1 — ข้อความที่ถูกต้อง: "([{}])"

```
อ่าน '('  →  push          →  stack: ['(']
อ่าน '['  →  push          →  stack: ['(', '[']
อ่าน '{'  →  push          →  stack: ['(', '[', '{']
อ่าน '}'  →  pop '{' → ตรงกับ '}' ✓   →  stack: ['(', '[']
อ่าน ']'  →  pop '[' → ตรงกับ ']' ✓   →  stack: ['(']
อ่าน ')'  →  pop '(' → ตรงกับ ')' ✓   →  stack: []

สิ้นสุดข้อความ → stack ว่างเปล่า → สรุปว่า True (ถูกต้อง)
```

- ความซับซ้อน algorithm นี้มีค่าเป็น O(n) โดยที่ n คือความยาวของข้อความที่ตรวจสอบ
- เพราะแต่ละตัวอักษรถูกอ่านเพียงครั้งเดียว และการดำเนินการกับ stack ในแต่ละครั้ง (ไม่ว่าจะเป็น push หรือ pop) มีความซับซ้อนเป็น O(1)

ตัวอย่างการไล่ทำงานทีละขั้นตอน

ตัวอย่างที่ 2 — ข้อความที่ไม่ถูกต้อง: "([)]"

```
อ่าน '('  →  push          →  stack: ['(']
อ่าน '['  →  push          →  stack: ['(', '[']
อ่าน '{'  →  push          →  stack: ['(', '[', '{']
อ่าน '}'  →  pop '{' → ตรงกับ '}' ✓   →  stack: ['(', '[']
อ่าน ']'  →  pop '[' → ตรงกับ ']' ✓   →  stack: ['(']
อ่าน ')'  →  pop '(' → ตรงกับ ')' ✓   →  stack: []

สิ้นสุดข้อความ → stack ว่างเปล่า → สรุปว่า True (ถูกต้อง)
```

- **แม้ว่าข้อความนี้จะมีวงเล็บเปิดและวงเล็บปิดครบจำนวนเท่ากันทุกชนิด**
  - **มี (และ) อย่างละหนึ่ง มี [และ] อย่างละหนึ่ง**
- **แต่ลำดับการปิดผิดไปจากกฎ เพราะ) ที่ปรากฏขึ้นพยายามปิดวงเล็บ [ที่ยังไม่ควรถูกปิดในจังหวะนั้น**
  - **ทั้งที่ [เพิ่งถูกเปิดขึ้นมาล่าสุดและควรรอให้ถูกปิดด้วย] ก่อนเสมอตามหลัก LIFO**

หลักการ Undo/Redo

Undo/Redo Mechanism

- ระบบ Undo/Redo ที่พบได้ในโปรแกรมแก้ไขข้อความแทบทุกตัว เป็นอีกหนึ่งตัวอย่างคลาสสิกของการนำ Stack ไปประยุกต์ใช้งานจริง
  - ต่างจาก bracket matching คือ ระบบนี้ต้องใช้ Stack สองตัวทำงานร่วมกัน
- บทบาทของ Stack ทั้งสองตัว
  - Stack ตัวแรกเรียกว่า undo_stack ทำหน้าที่เก็บสถานะ (state) ก่อนหน้าของข้อมูลไว้ทุกครั้งที่มีการเปลี่ยนแปลงเกิดขึ้น เพื่อให้สามารถย้อนกลับไปยังสถานะนั้นได้เมื่อผู้ใช้ต้องการ Undo
  - ส่วน Stack ตัวที่สองเรียกว่า redo_stack ทำหน้าที่เก็บสถานะที่เพิ่งถูก Undo ไป เพื่อให้สามารถนำกลับมาใช้ใหม่ได้หากผู้ใช้ต้องการ Redo

กฎการทำงานระบบ Undo/Redo

- ข้อที่ 1 ทุกครั้งที่ผู้ใช้ทำ action ใหม่ (เช่น พิมพ์ตัวอักษรเพิ่ม หรือลบตัวอักษร) ให้ push สถานะเดิมของข้อมูล (ก่อนการเปลี่ยนแปลงครั้งนี้) ลงใน undo_stack ก่อน แล้วจึงค่อยเปลี่ยนแปลงสถานะจริง
- ข้อที่ 2 เมื่อผู้ใช้กด Undo ให้ pop สถานะล่าสุดออกจาก undo_stack เพื่อนำมาใช้เป็นสถานะปัจจุบัน แต่ก่อนที่จะทำเช่นนั้น ต้อง push สถานะปัจจุบัน (ก่อนถูก undo) ลงใน redo_stack เก็บไว้ก่อน เพื่อให้สามารถกลับมาที่สถานะนี้ได้อีกครั้งหากมีการกด Redo ในภายหลัง
- ข้อที่ 3 เมื่อผู้ใช้กด Redo ให้ทำในทางตรงกันข้ามกับข้อที่ 2 คือ pop สถานะออกจาก redo_stack มาใช้เป็นสถานะปัจจุบัน โดย push สถานะปัจจุบัน (ก่อนถูก redo) ลงใน undo_stack เก็บไว้ก่อนเช่นกัน
- ข้อที่ 4 ทุกครั้งที่ผู้ใช้ทำ action ใหม่ที่ไม่ใช่การ Undo หรือ Redo จะต้อง เคลียร์ redo_stack ทิ้งทั้งหมด เพราะ “อนาคต” ที่เคยสามารถ Redo กลับไปได้นั้น ได้ถูกแทนที่ด้วยเส้นทางใหม่ที่ผู้ใช้เพิ่งสร้างขึ้นแล้ว
  - การไม่เคลียร์ redo_stack ในจุดนี้จะทำให้เกิดพฤติกรรมที่ขัดกับความคาดหวังของผู้ใช้

กฎการทำงานระบบ Undo/Redo

ภาพการทำงานของกฎทั้งสี่ข้อร่วมกัน ลองพิจารณาลำดับเหตุการณ์ต่อไปนี้ โดยเริ่มจากข้อความว่างเปล่า

```
เริ่มต้น          : text = ""

พิมพ์ "Hello"     : undo=[""]              redo=[]
text="Hello"
พิมพ์ " World"    : undo=["", "Hello"]     redo=[]
text="Hello World"

Undo              : undo=[""]                redo=["Hello World"]
text="Hello"
Undo              : undo=[]                  redo=["Hello World","Hello"]
text=""

Redo              : undo=[""]                redo=["Hello World"]
text="Hello"

พิมพ์ "!!!"       : undo=["", "Hello"]     redo=[]
text="Hello!!!"
                                                ↑ redo_stack ถูกเคลียร์ทิ้งตามกฎข้อที่ 4
```

- หลังจากที่ผู้ใช้กด Redo กลับไปที่ "Hello" แล้วพิมพ์ "!!!" เพิ่มเข้าไปเป็น action ใหม่ ค่า redo_stack ที่เคยมี "Hello World" ค้างอยู่จะถูกเคลียร์ทิ้งทันที เพราะเส้นทางเดิมที่เคยนำไปสู่ "Hello World" ได้ถูกแทนที่ด้วยเส้นทางใหม่ที่นำไปสู่ "Hello!!!" แล้ว
- หากมีการกด Redo อีกครั้งหลังจากจุดนี้ จะไม่มีสิ่งใดให้ Redo ได้อีกเลย

โค้ดฉบับสมบูรณ์ของ Node กับ Stack

คลาส Node

```python
class Node:
    """Node สำหรับ Linked List-based Stack (เหมือนกับ Node ในสัปดาห์ที่ 10 ทุกประการ)"""

    def __init__(self, data):
        self.data = data
        self.next = None    # ชี้ไปยัง node ถัดไป (ด้านล่างใน stack)
```

คลาส Stack

__init__ ตั้งค่า _top = None และ _size = 0 เสมอ
เพราะ Stack ที่เพิ่งถูกสร้างขึ้น

```
class Stack:
    """
    Stack implementation โดยใช้ Linked List เป็นโครงสร้างเบื้องหลัง
    - top ของ stack = head ของ linked list
    - push = prepend (เพิ่มที่หัว)
    - pop  = remove head
    ทุก operation เป็น O(1) อย่างแท้จริง (ไม่มี amortized)
    """

    def __init__(self):
        self._top = None     # เทียบเท่ากับ head ของ linked list
        self._size = 0
```

- pop เริ่มต้นด้วยการตรวจสอบ is_empty() ก่อนเสมอเพื่อป้องกันไม่ให้เกิด AttributeError จากการพยายามเข้าถึง self._top.data ในขณะที่ _top เป็น None
- หากตรวจพบว่า stack ว่างเปล่า จะ raise IndexError พร้อมข้อความว่า “พยายาม pop จาก stack ที่ว่างเปล่า”

push ใช้รูปแบบเดียวกับ prepend ของ SinglyLinkedList ทุกประการ
ให้ Node ใหม่ชี้ไปยัง _top เดิมก่อน (ขั้นที่ 1)
แล้วจึงอัปเดต _top ให้ชี้มาที่ Node ใหม่ (ขั้นที่ 2)

```
def push(self, item) -> None:
    """เพิ่ม item ลงบน top ของ stack -- O(1)"""
    new_node = Node(item)
    new_node.next = self._top    # ขั้นที่ 1: ชี้ไปยัง top เดิมก่อน
    self._top = new_node         # ขั้นที่ 2: อัปเดต top
    self._size += 1
```

```
def pop(self):
    """
    นำ item ออกจาก top และ return -- O(1)
    Raises: IndexError ถ้า stack ว่าง
    """
    if self.is_empty():
        raise IndexError("pop from empty stack")
    data = self._top.data
    self._top = self._top.next    # เลื่อน top ลงไปหนึ่ง node
    self._size -= 1
    return data
```

คลาส Stack (2)

peek มีโครงสร้างคล้าย pop มาก แตกต่างกันตรงที่ peek ไม่เปลี่ยนแปลงค่า _top เลย เป็นเพียงการอ่านค่า data ของ Node ที่ _top ชี้อยู่แล้วคืนค่ากลับไปเท่านั้น

```
def peek(self):
    """
    ดู item บน top โดยไม่นำออก -- O(1)
    Raises: IndexError ถ้า stack ว่าง
    """
    if self.is_empty():
        raise IndexError("peek from empty stack")
    return self._top.data
```

is_empty ตรวจสอบเพียงว่า _top เป็น None หรือไม่

```
def is_empty(self) -> bool:
    """ตรวจสอบว่า stack ว่างหรือไม่ -- O(1)"""
    return self._top is None

def __len__(self) -> int:
    return self._size
```

```
def __str__(self) -> str:
    """แสดง stack จาก top ไปยัง bottom"""
    if self.is_empty():
        return "Stack: [empty]"
    elements = []
    current = self._top
    while current:
        elements.append(str(current.data))
        current = current.next
    return "TOP → " + " → ".join(elements) + " → BOTTOM"
```

- __str__ ใช้ traversal pattern พื้นฐานในการไล่เก็บค่าจากทุก Node เริ่มจาก _top ไปจนสุด (current is None)
- ลำดับการแสดงผลจะเริ่มจาก top ไปยัง bottom ซึ่งตรงกับลำดับการ traverse ตามธรรมชาติของ Linked List (จาก head ไปยัง tail) พอดี

ArrayStack แล: Bracket Matching

ArrayStack

- Stack สามารถ implement ได้จาก Python List โดยตรงเช่นกัน โดยกำหนดให้ตำแหน่ง top ตรงกับตำแหน่งท้ายสุดของ list

```
class ArrayStack:
    """
    Stack โดยใช้ Python List เป็นโครงสร้างเบื้องหลัง
    push = list.append()  -> O(1) amortized
    pop  = list.pop()     -> O(1)
    ทั้งคู่ไม่ต้อง traverse ใดๆ เพราะจัดการที่ปลายท้ายของ list โดยตรง
    """

    def __init__(self):
        self._data = []

    def push(self, item) -> None:
        self._data.append(item)      # append ที่ท้าย list = top ของ stack

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._data.pop()      # pop ที่ท้าย list = O(1)
```

```
def peek(self):
    if self.is_empty():
        raise IndexError("peek from empty stack")
    return self._data[-1]         # ดูตัวสุดท้าย

def is_empty(self) -> bool:
    return len(self._data) == 0

def __len__(self) -> int:
    return len(self._data)

def __str__(self) -> str:
    if self.is_empty():
        return "Stack: [empty]"
    elements = list(reversed(self._data))
    return "TOP → " + " → ".join(str(x) for x in elements) + " → BOTTOM"
```

- __str__ ต้องเรียก reversed(self._data) ก่อนนำมาต่อกันเป็น string เพราะใน ArrayStack ตำแหน่ง top คือตำแหน่งท้ายสุดของ list (index -1)
- list เก็บข้อมูลเรียงจาก index 0 ไปหา index สุดท้ายตามธรรมชาติ หากไม่กลับลำดับก่อน การแสดงผลจะเรียงจาก bottom ไปหา top ซึ่งขัดกับรูปแบบการแสดงผลที่ต้องการ

Application of stack

Bracket Matching (is_balanced)

```python
from stack import Stack


def is_balanced(text: str) -> bool:
    """
    ตรวจสอบว่าวงเล็บใน text ถูกเปิดและปิดครบถ้วนถูกต้องหรือไม่
    รองรับวงเล็บสามชนิด: (), [], {}
    ตัวอักษรอื่นทั้งหมดที่ไม่ใช่วงเล็บจะถูกละเว้น ไม่มีผลต่อการตรวจสอบ

    Complexity: O(n) เพราะอ่านแต่ละตัวอักษรเพียงครั้งเดียว
                และแต่ละครั้งทำ operation กับ stack ที่เป็น O(1)
    """
    stack = Stack()
    # closing bracket -> opening bracket ที่ควรจับคู่
    matching = {')': '(', ']': '[', '}': '{'}

    opening = set('([{')

    for char in text:
        if char in opening:
            stack.push(char)
        elif char in matching:
            if stack.is_empty():
                return False          # closing ที่ไม่มี opening คู่กันเลย
            if stack.pop() != matching[char]:
                return False          # opening ที่ pop ออกมาไม่ตรงกับ closing นี้

    return stack.is_empty()           # True ถ้าไม่มี opening ค้างอยู่เลย
```

- การใช้ dictionary matching
  - ออกแบบให้ key เป็นวงเล็บปิด และ value เป็นวงเล็บเปิดที่ควรจับคู่ด้วย
  - การออกแบบในทิศทางนี้ (จาก closing ไปหา opening) มีเหตุผลสำคัญคือ เมื่ออ่านพบวงเล็บปิดตัวหนึ่ง สิ่งที่ต้องการรู้ทันทีคือ “วงเล็บเปิดชนิดใดที่ควรจะอยู่คู่กับมัน” เพื่อนำไปเทียบกับค่าที่ pop ออกมาจาก stack ได้โดยตรงในบรรทัดเดียว (stack.pop() != matching[char])
  - หากออกแบบ dictionary ในทิศทางกลับกัน (จาก opening ไปหา closing) จะต้องเขียนโค้ดเพิ่มอีกขั้นตอนหนึ่งในการกลับทิศทางการค้นหา

Bracket Matching (is_balanced)

```python
from stack import Stack


def is_balanced(text: str) -> bool:
    """
    ตรวจสอบว่าวงเล็บใน text ถูกเปิดและปิดครบถ้วนถูกต้องหรือไม่
    รองรับวงเล็บสามชนิด: (), [], {}
    ตัวอักษรอื่นทั้งหมดที่ไม่ใช่วงเล็บจะถูกละเว้น ไม่มีผลต่อการตรวจสอบ

    Complexity: O(n) เพราะอ่านแต่ละตัวอักษรเพียงครั้งเดียว
                และแต่ละครั้งทำ operation กับ stack ที่เป็น O(1)
    """
    stack = Stack()
    # closing bracket -> opening bracket ที่ควรจับคู่
    matching = {')': '(', ']': '[', '}': '{'}

    opening = set('([{')

    for char in text:
        if char in opening:
            stack.push(char)
        elif char in matching:
            if stack.is_empty():
                return False          # closing ที่ไม่มี opening คู่กันเลย
            if stack.pop() != matching[char]:
                return False          # opening ที่ pop ออกมาไม่ตรงกับ closing นี้

    return stack.is_empty()           # True ถ้าไม่มี opening ค้างอยู่เลย
```

- การใช้ set('([{') แทน list หรือ string ธรรมดา
  - เพื่อให้การตรวจสอบ char in opening มีความซับซ้อนเป็น O(1) แทนที่จะเป็น O(k) ซึ่ง k คือจำนวนชนิดของวงเล็บเปิด
  - แม้ในกรณีนี้ k มีค่าเพียง 3 ทำให้ความแตกต่างไม่มีนัยสำคัญในทางปฏิบัติ
  - แต่การใช้ set ยังคงเป็นแนวปฏิบัติที่ดีเพราะสื่อความหมายชัดเจนว่ากำลังตรวจสอบการเป็นสมาชิกของกลุ่ม ไม่ใช่การค้นหาลำดับ

Bracket Matching (is_balanced)

```python
from stack import Stack


def is_balanced(text: str) -> bool:
    """
    ตรวจสอบว่าวงเล็บใน text ถูกเปิดและปิดครบถ้วนถูกต้องหรือไม่
    รองรับวงเล็บสามชนิด: (), [], {}
    ตัวอักษรอื่นทั้งหมดที่ไม่ใช่วงเล็บจะถูกละเว้น ไม่มีผลต่อการตรวจสอบ

    Complexity: O(n) เพราะอ่านแต่ละตัวอักษรเพียงครั้งเดียว
                และแต่ละครั้งทำ operation กับ stack ที่เป็น O(1)
    """
    stack = Stack()
    # closing bracket -> opening bracket ที่ควรจับคู่
    matching = {')': '(', ']': '[', '}': '{'}

    opening = set('([{')

    for char in text:
        if char in opening:
            stack.push(char)
        elif char in matching:
            if stack.is_empty():
                return False          # closing ที่ไม่มี opening คู่กันเลย
            if stack.pop() != matching[char]:
                return False          # opening ที่ pop ออกมาไม่ตรงกับ closing นี้

    return stack.is_empty()           # True ถ้าไม่มี opening ค้างอยู่เลย
```

- เงื่อนไข if stack.is_empty(): return False เป็นการนำกฎข้อที่ 1 มาใช้
  - หากพบวงเล็บปิดในขณะที่ stack ว่างเปล่าอยู่พอดี แสดงว่าไม่เคยมีวงเล็บเปิดชนิดใดถูกเปิดค้างไว้เลย
  - ไม่มีทางที่วงเล็บปิดตัวนี้จะถูกต้อง คืนค่า False ทันทีโดยไม่ต้องอ่านข้อความที่เหลือต่อไปอีก

Bracket Matching (is_balanced)

```python
from stack import Stack


def is_balanced(text: str) -> bool:
    """
    ตรวจสอบว่าวงเล็บใน text ถูกเปิดและปิดครบถ้วนถูกต้องหรือไม่
    รองรับวงเล็บสามชนิด: (), [], {}
    ตัวอักษรอื่นทั้งหมดที่ไม่ใช่วงเล็บจะถูกละเว้น ไม่มีผลต่อการตรวจสอบ

    Complexity: O(n) เพราะอ่านแต่ละตัวอักษรเพียงครั้งเดียว
                และแต่ละครั้งทำ operation กับ stack ที่เป็น O(1)
    """
    stack = Stack()
    # closing bracket -> opening bracket ที่ควรจับคู่
    matching = {')': '(', ']': '[', '}': '{'}

    opening = set('([{')

    for char in text:
        if char in opening:
            stack.push(char)
        elif char in matching:
            if stack.is_empty():
                return False          # closing ที่ไม่มี opening คู่กันเลย
            if stack.pop() != matching[char]:
                return False          # opening ที่ pop ออกมาไม่ตรงกับ closing นี้

    return stack.is_empty()           # True ถ้าไม่มี opening ค้างอยู่เลย
```

- เงื่อนไข if stack.pop() != matching[char]: return False นำกฎข้อที่ 2 มาใช้
  - การเรียก stack.pop() ในบรรทัดนี้ได้นำวงเล็บเปิดตัวบนสุดออกจาก stack ไปแล้วไม่ว่าผลการเปรียบเทียบจะเป็นอย่างไรก็ตาม ซึ่งเป็นพฤติกรรมที่ถูกต้อง
  - ไม่ว่าวงเล็บที่ pop ออกมาจะตรงกับ char หรือไม่ วงเล็บนั้นก็ไม่ควรอยู่ใน stack อีกต่อไปแล้ว
    - หากตรงกัน มันถูกจับคู่และปิดเรียบร้อยแล้ว
    - หากไม่ตรงกัน โปรแกรมกำลังจะคืนค่า False และยุติการทำงาน

Bracket Matching (is_balanced)

```python
from stack import Stack


def is_balanced(text: str) -> bool:
    """
    ตรวจสอบว่าวงเล็บใน text ถูกเปิดและปิดครบถ้วนถูกต้องหรือไม่
    รองรับวงเล็บสามชนิด: (), [], {}
    ตัวอักษรอื่นทั้งหมดที่ไม่ใช่วงเล็บจะถูกละเว้น ไม่มีผลต่อการตรวจสอบ

    Complexity: O(n) เพราะอ่านแต่ละตัวอักษรเพียงครั้งเดียว
                และแต่ละครั้งทำ operation กับ stack ที่เป็น O(1)
    """
    stack = Stack()
    # closing bracket -> opening bracket ที่ควรจับคู่
    matching = {')': '(', ']': '[', '}': '{'}

    opening = set('([{')

    for char in text:
        if char in opening:
            stack.push(char)
        elif char in matching:
            if stack.is_empty():
                return False          # closing ที่ไม่มี opening คู่กันเลย
            if stack.pop() != matching[char]:
                return False          # opening ที่ pop ออกมาไม่ตรงกับ closing นี้

    return stack.is_empty()           # True ถ้าไม่มี opening ค้างอยู่เลย
```

- return stack.is_empty() ที่บรรทัดสุดท้าย นำกฎข้อที่ 3 มาใช้
  - หากอ่านข้อความจนจบแล้ว stack ยังไม่ว่างเปล่า แสดงว่ามีวงเล็บเปิดบางตัวที่ไม่เคยถูกปิดเลย

- ฟังก์ชันนี้ไม่ได้ตรวจสอบตัวอักษรที่ไม่ใช่วงเล็บเลยแม้แต่น้อย
  - ตัวอักษรใดๆ ที่ไม่ตรงกับเงื่อนไข opening หรือ matching จะถูกข้ามผ่านไปโดยอัตโนมัติ
  - นี่คือเหตุผลที่ทำให้ is_balanced("hello (world)") และ is_balanced("def foo(x, y): return [x, y]") ให้ผลลัพธ์ True ได้อย่างถูกต้อง
    - แม้ข้อความจะมีตัวอักษรอื่นปะปนอยู่จำนวนมากก็ตาม

TextEditor (Undo/Redo)

```python
from stack import Stack


class TextEditor:
    """
    Text editor อย่างง่ายที่รองรับ Undo และ Redo
    กลยุทธ์ที่ใช้: เก็บ snapshot ของข้อความทั้งหมดไว้ในแต่ละ step (เรียบง่ายที่สุด)

    _undo_stack เก็บ state ก่อนหน้า (ใช้เมื่อกด Undo เพื่อย้อนกลับ)
    _redo_stack เก็บ state ที่ถูก Undo ไปแล้ว (ใช้เมื่อกด Redo เพื่อกลับมาใหม่)
    """

    def __init__(self):
        self._text = ""
        self._undo_stack = Stack()
        self._redo_stack = Stack()

    def type_text(self, text: str) -> None:
        """พิมพ์ข้อความต่อท้าย"""
        self._undo_stack.push(self._text)      # ขั้นที่ 1: บันทึก state ก่อนเปลี่ยน
        self._redo_stack = Stack()     # ขั้นที่ 2: action ใหม่ -> ลบ redo history ทิ้ง
        self._text += text                     # ขั้นที่ 3: เปลี่ยน state จริง

    def delete_last(self) -> None:
        """ลบตัวอักษรตัวสุดท้ายออกจากข้อความ"""
        if not self._text:
            return
        self._undo_stack.push(self._text)
        self._redo_stack = Stack()
        self._text = self._text[:-1]
```

- type_text และ delete_last มีโครงสร้างเหมือนกันทุกประการ
- ทำตามลำดับสามขั้นตอนเดียวกัน (บันทึก state เดิม, เคลียร์ redo, เปลี่ยน state จริง)
- ความคล้ายคลึงกันนี้สะท้อนให้เห็นว่าทุก action ที่เปลี่ยนแปลงข้อมูลของผู้ใช้ (ไม่ว่าจะเป็นการเพิ่มหรือลบ) ต้องปฏิบัติตามกฎเดียวกันเสมอ
- หากในอนาคตมีการเพิ่ม method ใหม่ที่เปลี่ยนแปลง _text เช่น replace_all หรือ clear method เหล่านั้นก็จำเป็นต้องปฏิบัติตามรูปแบบสามขั้นตอนนี้เช่นกัน
    - ไม่เช่นนั้นระบบ Undo/Redo จะทำงานผิดพลาดทันที

TextEditor (Undo/Redo)

```python
from stack import Stack


class TextEditor:
    """
    Text editor อย่างง่ายที่รองรับ Undo และ Redo
    กลยุทธ์ที่ใช้: เก็บ snapshot ของข้อความทั้งหมดไว้ในแต่ละ step (เรียบง่ายที่สุด)

    _undo_stack เก็บ state ก่อนหน้า (ใช้เมื่อกด Undo เพื่อย้อนกลับ)
    _redo_stack เก็บ state ที่ถูก Undo ไปแล้ว (ใช้เมื่อกด Redo เพื่อกลับมาใหม่)
    """

    def __init__(self):
        self._text = ""
        self._undo_stack = Stack()
        self._redo_stack = Stack()

    def type_text(self, text: str) -> None:
        """พิมพ์ข้อความต่อท้าย"""
        self._undo_stack.push(self._text)      # ขั้นที่ 1: บันทึก state ก่อนเปลี่ยน
        self._redo_stack = Stack()     # ขั้นที่ 2: action ใหม่ -> ลบ redo history ทิ้ง
        self._text += text                     # ขั้นที่ 3: เปลี่ยน state จริง

    def delete_last(self) -> None:
        """ลบตัวอักษรตัวสุดท้ายออกจากข้อความ"""
        if not self._text:
            return
        self._undo_stack.push(self._text)
        self._redo_stack = Stack()
        self._text = self._text[:-1]
```

- **self._redo_stack = Stack()**
- วิธีการ “เคลียร์” stack ที่ทำโดยการสร้าง Stack() ตัวใหม่ขึ้นมาทับตัวเดิมไปเลย แทนที่จะเขียนลูป while not redo_stack.is_empty(): redo_stack.pop() ซึ่งได้ผลลัพธ์เหมือนกันแต่ใช้เวลานานกว่าโดยไม่จำเป็น
- เพราะ Stack ตัวเดิมที่ไม่มีตัวแปรใดอ้างอิงถึงอีกต่อไปจะถูก Python จัดการเก็บกวาดหน่วยความจำให้อัตโนมัติอยู่แล้ว

TextEditor (Undo/Redo)

```
def undo(self) -> None:
    """ย้อนกลับ action ล่าสุด"""
    if self._undo_stack.is_empty():
        return
    self._redo_stack.push(self._text)      # เก็บ state ปัจจุบันไว้ก่อนเปลี่ยน
    self._text = self._undo_stack.pop()  # นำ state เก่ากลับมาใช้เป็น state ปัจจุบัน

def redo(self) -> None:
    """ทำ action ที่ถูก Undo ไปซ้ำอีกครั้ง"""
    if self._redo_stack.is_empty():
        return
    self._undo_stack.push(self._text)      # เก็บ state ปัจจุบันไว้ก่อนเปลี่ยน
    self._text = self._redo_stack.pop()    # นำ state ที่เคยถูก undo กลับมาใช้

def get_text(self) -> str:
    return self._text

def __str__(self) -> str:
    return (f"Text    : {self._text!r}\n"
            f"Undos   : {len(self._undo_stack)} available\n"
            f"Redos   : {len(self._redo_stack)} available")
```

- undo เริ่มต้นด้วยการตรวจสอบว่า _undo_stack ว่างเปล่าหรือไม่
  - หากว่างเปล่าแสดงว่าไม่มี state ใดให้ย้อนกลับไปได้อีกแล้ว จึง return ออกจาก method ทันทีโดยไม่ทำอะไร
- การเลือกให้ undo เป็น no-op แทนที่จะ raise error สะท้อนถึงการออกแบบในระดับ application
  - ผู้ใช้โปรแกรมแก้ไขข้อความที่กด Ctrl+Z ซ้ำเกินความจำเป็นไม่ควรเห็นโปรแกรม crash แต่ควรที่จะไม่มีอะไรเกิดขึ้นเพิ่มเติมเท่านั้น
- จุดนี้แสดงให้เห็นถึงความแตกต่างที่สำคัญระหว่างพฤติกรรมของ Stack ในระดับ ADT
  - ควร raise error เมื่อผิดกฎ เพื่อบังคับให้ผู้เขียนโค้ดตรวจสอบก่อนเรียกใช้
  - พฤติกรรมของ method ในระดับ application ที่ประกอบขึ้นจาก Stack อีกที ควรไม่มีอะไรเกิดขึ้นก็พอ

TextEditor (Undo/Redo)

```
def undo(self) -> None:
    """ย้อนกลับ action ล่าสุด"""
    if self._undo_stack.is_empty():
        return
    self._redo_stack.push(self._text)      # เก็บ state ปัจจุบันไว้ก่อนเปลี่ยน
    self._text = self._undo_stack.pop()  # นำ state เก่ากลับมาใช้เป็น state ปัจจุบัน

def redo(self) -> None:
    """ทำ action ที่ถูก Undo ไปซ้ำอีกครั้ง"""
    if self._redo_stack.is_empty():
        return
    self._undo_stack.push(self._text)      # เก็บ state ปัจจุบันไว้ก่อนเปลี่ยน
    self._text = self._redo_stack.pop()    # นำ state ที่เคยถูก undo กลับมาใช้

def get_text(self) -> str:
    return self._text

def __str__(self) -> str:
    return (f"Text    : {self._text!r}\n"
            f"Undos   : {len(self._undo_stack)} available\n"
            f"Redos   : {len(self._redo_stack)} available")
```

- redo มีโครงสร้างสมมาตรกับ undo ทุกประการ เพียงแค่สลับบทบาทระหว่าง _undo_stack กับ _redo_stack

- __str__ แสดงจำนวนรายการที่มีอยู่ใน _undo_stack และ _redo_stack ผ่าน len() โดยไม่จำเป็นต้องรู้เนื้อหาภายในของแต่ละ state
  - เพราะ Stack ไม่มี operation ให้เข้าถึงข้อมูลตรงกลางหรือดูข้อมูลทั้งหมดพร้อมกันได้อยู่แล้ว
  - การแสดงเพียง “จำนวนที่มีอยู่” เป็นข้อมูลสูงสุดที่สามารถเปิดเผยออกมาได้โดยไม่ละเมิดหลักการ encapsulation ของ Stack